

LeetCode Hot 100

Go + Python 双语言快速复习手册

100 题模式化分类 · 核心思路 · 易错点 · 扩展题 · 关键题型示意图 · 60 天路径

目标	60 天完成 Hot 100 两轮复习，并能在 Go / Python 之间快速切换实现。
方法	第一轮按 Pattern 学母题；第二轮语言互换；最后 10 天做限时混合题与 Mock。
重点	不背 100 份独立答案，而是掌握约 14 个可迁移模式：哈希、双指针、滑窗、前缀和、链表、树、图、回溯、二分、单调栈、堆、贪心、DP、技巧。

推荐使用方式

每天 60-90 分钟：20 分钟复习模板与示意图，30-50 分钟做当日题，10-20 分钟把其中 1 道用另一种语言重写。每题都要能回答：为什么这个模式适用？核心不变量是什么？边界在哪里？还能扩展成什么题？

说明：Hot 100 题单在平台展示形式可能调整；本手册采用当前广泛使用的 LeetCode Hot 100 经典 100 题集合，并按复习模式重新分组。

一、先建立模式地图

模式	代表题	看到什么信号就想到它
1. 哈希 / 集合	#1 两数之和、#49 字母异位词分组、#128 最长连续序列	先问：我需要 O(1) 查什么？值、计数、索引还是状态。哈希的核心不是“会用 map”，而是选对 key/value 与更新时机。
2. 双指针	#283 移动零、#11 盛最多水的容器、#15 三数之和、#42 接雨水	一类是同向快慢指针，另一类是左右夹逼。关键是能说出“为什么移动这一侧不会漏解”。
3. 滑动窗口 / 前缀和	#3 无重复字符的最长子串、#438 找到字符串中所有字母异位词、#560 和为 K 的子数组、#239 滑动窗口最大值 …	滑动适合连续区间且窗口变化可增量维护；前缀和适合快速表达区间和，配哈希处理“历史状态”。
4. 数组 / 区间 / 矩阵	#53 最大子数组和、#56 合并区间、#189 轮转数组、#238 除自身以外数组的乘积 …	数组题常考原地修改、前后缀、区间排序和边界模拟。先判断能否通过排序/索引映射消除复杂状态。
5. 链表	#160 相交链表、#206 反转链表、#234 回文链表、#141 环形链表 …	链表题先画节点和指针。dummy 解决头结点特判，快慢指针解决距离/环，反转模板解决一整组题。
6. 二叉树	#94 二叉树的中序遍历、#104 二叉树的最大深度、#226 翻转二叉树、#101 对称二叉树 …	树题首先判断 DFS 还是 BFS；DFS 再区分“自顶向下携带状态”还是“后序从子树收集信息”。
7. 图 / Trie	#200 岛屿数量、#994 腐烂的橘子、#207 课程表、#208 实现 Trie	图题重点是建模：节点是谁、边是什么、visited 何时标记。最短传播时间通常 BFS，有依赖关系通常拓扑排序。
8. 回溯	#46 全排列、#78 子集、#17 电话号码的字母组合、#39 组合总和 …	统一模板：选择列表 -> 做选择 -> 递归 -> 撤销选择；最重要的是 start/used/剪枝条件。
9. 二分搜索	#35 搜索插入位置、#74 搜索二维矩阵、#34 在排序数组中查找元素的第一个和最后一个位置、#33 搜索旋转排序数组 …	不要死记题型，统一成“在单谓词上找边界”。推荐先熟 lower_bound，再处理旋转数组的局部有序性。
10. 栈 / 单调栈	#20 有效的括号、#155 最小栈、#394 字符串解码、#739 每日温度 …	普通栈处理嵌套与匹配；单调栈解决“下一个更大/更小”和“以当前元素为最值能扩到哪里”。
11. 堆	#215 数组中的第 K 个最大元素、#347 前 K 个高频元素、#295 数据流的中位数	Top K 用大小 k 的堆；流式中位数用双堆平衡。Go 的 heap.Interface 和 Python heapq 都要熟。
12. 贪心	#121 买卖股票的最佳时机、#55 跳跃游戏、#45 跳跃游戏 II、#763 划分字母区间	贪心不是“看起来最优”，而是维护一个可证明的不变量或覆盖范围。
13. 动态规划	#70 爬楼梯、#118 杨辉三角、#198 打家劫舍、#279 完全平方数 …	DP 四步：状态、选择/转移、初始化、遍历顺序。背包题再额外检查容量循环方向。
14. 高频技巧	#136 只出现一次的数字、#169 多数元素、#75 颜色分类、#31 下一个排列 …	位运算、投票、三路分区、字典序、Floyd 都属于“面试一眼识别”的技巧题，适合最后阶段集中记忆。

二、Go / Python 刷题语法快速切换

算法思路相同，真正拖慢速度的往往是语言细节。下面这些差异建议在第 1 周就练成肌肉记忆。

场景	Go	Python	常见坑
哈希/集合	map[K]V ; v,ok := m[k] ; set 常用 map[T]struct{}	dict / set ; get ; Counter	Go 零值与 ok ; Python key 必须 hashable
队列	slice + head index 更高效 ; 小题也可 q=q[1:]	collections.deque	Python list.pop(0) 是 O(n)
堆	container/heap 实现 heap.Interface	heapq 默认小顶堆，大顶堆常用负数	比较方向与类型转换
排序	sort.Ints / sort.Slice	sort / sorted(key=...)	稳定性、原地/返回值差异
字符串	ASCII 可直接 byte ; Unicode 用 []rune	str Unicode ; 切片方便	Go len(string) 是字节数
递归	递归闭包先 var dfs func(...) 再赋值	def 内嵌函数 ; nonlocal	深递归 Python 受 recursion limit 影响
复制 path	append([]int(nil), path...)	path.copy() / path[:]	不复制会导致结果共享同一底层对象
整数	int 范围与平台相关 ; 需要时 int64	任意精度 int	乘积/前缀和在 Go 里关注溢出

1. 哈希 / 集合

先问：我需要 O(1) 查什么？值、计数、索引还是状态。哈希的核心不是“会用 map”，而是选对 key/value 与更新时机。



双语言核心模板

Go	Python
<pre>// Go: hash lookup m := map[int]int{} for i, x := range nums { if j, ok := m[target-x]; ok { return []int{j,i} } m[x] = i }</pre>	<pre># Python: hash lookup pos = {} for i, x in enumerate(nums): if target-x in pos: return [pos[target-x], i] pos[x] = i</pre>

题号	题目	一句话识别
#1	两数之和	遍历时查 target-x 是否已出现；先查后放避免同一元素重复使用。
#49	字母异位词分组	把每个字符串映射到稳定签名：排序串或 26 字母计数 tuple/array。
#128	最长连续序列	把数放 set，只从没有前驱 x-1 的数开始向右扩展。

2. 双指针

一类是同向快慢指针，另一类是左右夹逼。关键是能说出“为什么移动这一侧不会漏解”。

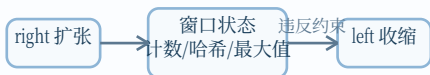


双语言核心模板

Go		Python
<pre>l, r := 0, len(nums)-1 for l < r { // 操作 l 和 r }</pre>		<pre>l, r = 0, len(nums)-1 while l < r: # 操作 l 和 r ...</pre>
题号	题目	一句话识别
#283	移动零	slow 指向下一个非零写入位置，fast 扫描；可交换或覆盖后补零。
#11	盛最多水的容器	左右夹逼，面积由短板决定，因此移动短板才可能变好。
#15	三数之和	排序后固定 i，对剩余区间双指针；严格跳过重复值。
#42	接雨水	双指针维护 leftMax/rightMax；较小一侧的水量已由该侧最大值决定。

3. 滑动窗口 / 前缀和

滑窗适合连续区间且窗口变化可增量维护；前缀和适合快速表达区间和，配哈希处理“历史状态”。



双语言核心模板

Go		Python
<pre>left := 0 cnt := map[byte]int{} for right := 0; right < len(s); right++ { cnt[s[right]]++ for invalid(cnt) { cnt[s[left]]--; left++ } }</pre>		<pre>left = 0 cnt = {} for right, ch in enumerate(s): cnt[ch] = cnt.get(ch, 0) + 1 while invalid(cnt): cnt[s[left]] -= 1 left += 1</pre>
题号	题目	一句话识别
#3	无重复字符的最长子串	right 扩张，记录字符最近位置或计数；重复时移动 left。
#438	找到字符串中所有字母异位词	固定长度窗口维护频次差；窗口达到 len(p) 后判断是否匹配。
#560	和为 K 的子数组	prefix[j]-prefix[i]=k，边扫边统计 prefix-k 出现次数。
#239	滑动窗口最大值	双端队列存下标，保持对应值单调递减；过期下标从队头弹出。
#76	最小覆盖子串	窗口扩张直到满足 need，再收缩 left 寻找最短合法窗口。

4. 数组 / 区间 / 矩阵

数组题常考原地修改、前后缀、区间排序和边界模拟。先判断能否通过排序/索引映射消除复杂状态。



双语言核心模板

Go	Python
<pre>l, r := 0, len(nums)-1 for l < r { // 区间操作 l 和 r }</pre>	<pre>l, r = 0, len(nums)-1 while l < r: # 区间操作 l 和 r ...</pre>

题号	题目	一句话识别
#53	最大子数组和	Kadane：以当前位置结尾的最优值 = $\max(\text{nums}[i], \text{prev} + \text{nums}[i])$ 。
#56	合并区间	按 start 排序，若当前 $\text{start} \leq \text{last.end}$ 就合并，否则新建区间。
#189	轮转数组	三次反转：整体反转，再反转前 k 和后 n-k。
#238	除自身以外数组的乘积	答案先存前缀积，再反向乘后缀积，避免除法。
#41	缺失的第一个正数	把值 x 放到下标 x-1 的位置，原地哈希；最后找首个 $\text{nums}[i] \neq i+1$ 。
#73	矩阵置零	用首行首列充当 marker，并单独记录首行/首列是否原本含 0。
#54	螺旋矩阵	维护 top/bottom/left/right 四边界，逐层收缩。
#48	旋转图像	转置 + 每行反转可得到顺时针 90 度旋转。
#240	搜索二维矩阵 II	从右上角开始：目标小则左移，大则下移。

5. 链表

链表题先画节点和指针。dummy 解决头结点特判，快慢指针解决距离/环，反转模板解决一整组题。



双语言核心模板

Go	Python
<pre>var prev *ListNode cur := head for cur != nil { next := cur.Next cur.Next = prev prev, cur = cur, next } return prev</pre>	<pre>prev, cur = None, head while cur: nxt = cur.next cur.next = prev prev, cur = cur, nxt return prev</pre>

题号	题目	一句话识别
#160	相交链表	两个指针走完自己的链再走对方链，路径长度对齐后在交点相遇。
#206	反转链表	prev/cur/next 三指针逐个反转 next。
#234	回文链表	快慢指针找中点，反转后半段，再双指针比较。
#141	环形链表	快慢指针；若有环最终相遇。
#142	环形链表 II	快慢相遇后，一指针回头部，两者同速再次相遇即环入口。
#21	合并两个有序链表	dummy + tail 每次接较小节点。
#2	两数相加	同步遍历两链表并维护 carry，直到两链与 carry 都为空。
#19	删除链表倒数第 N 个结点	dummy；fast 先走 n 步，再 fast/slow 同步直到 fast 到尾。
#24	两两交换链表中的节点	dummy 前驱 pre，每轮重连 a,b 三条边。
#25	K 个一组翻转链表	每次先确认剩余至少 k 个，再反转 [groupHead, groupTail] 并拼接。
#138	随机链表的复制	哈希 old->new 两遍建立；或原节点后插副本再拆分。
#148	排序链表	归并排序：快慢分割，递归排序左右，再 merge。
#23	合并 K 个升序链表	小顶堆维护每条链当前头节点；弹出最小并推进对应链。
#146	LRU 缓存	哈希定位节点 + 双向链表维护最近使用顺序；get/put 都 O(1)。

6. 二叉树

树题首先判断 DFS 还是 BFS；DFS 再区分“自顶向下携带状态”还是“后序从子树收集信息”。



双语言核心模板

Go	Python
<pre>var dfs func(*TreeNode) int dfs = func(root *TreeNode) int { if root == nil { return 0 } l, r := dfs(root.Left), dfs(root.Right) return 1 + max(l,r) }</pre>	<pre>def dfs(root): if not root: return 0 l, r = dfs(root.left), dfs(root.right) return 1 + max(l, r)</pre>

题号	题目	一句话识别
#94	二叉树的中序遍历	递归 left-root-right；迭代用栈一路压左。
#104	二叉树的最大深度	DFS 返回 1+max(left,right)，或 BFS 层数。
#226	翻转二叉树	递归交换左右子树。
#101	对称二叉树	递归比较 left.left vs right.right、left.right vs right.left。
#543	二叉树的直径	后序 DFS 返回高度，同时用 leftHeight+rightHeight 更新答案。
#102	二叉树的层序遍历	BFS，每层先读取 queue 当前长度。
#108	将有序数组转换为二叉搜索树	取中点为根，递归构造左右区间。
#98	验证二叉搜索树	DFS 传递合法 (low,high)；或中序必须严格递增。
#230	二叉搜索树中第 K 小的元素	BST 中序有序，遍历到第 k 个即可。
#199	二叉树的右视图	BFS 每层取最后一个；或 DFS 先右后左按深度首次访问。
#114	二叉树展开为链表	后序把左子树插到右侧，再把原右子树接到左子树最右节点。
#105	从前序与中序遍历序列构造二叉树	前序首元素是根，用 inorder 哈希定位左右子树规模递归切分。
#437	路径总和 III	DFS 路径前缀和 + 哈希统计 prefix-target；回溯时减计数。
#236	二叉树的最近公共祖先	若 root 是 p/q 或为空直接返；左右递归都非空则 root 为 LCA。
#124	二叉树中的最大路径和	后序返回“向上最大贡献”，负贡献截断为 0；全局答案考虑左右贡献+root。

7. 图 / Trie

图题重点是建模：节点是谁、边是什么、visited 何时标记。最短传播时间通常 BFS，有依赖关系通常拓扑排序。



双语言核心模板

Go	Python
<pre>q := []Node{start} visited[start] = true for len(q) > 0 { cur := q[0]; q = q[1:] for _, nxt := range neighbors(cur) { ... } }</pre>	<pre>from collections import deque q = deque([start]) visited = {start} while q: cur = q.popleft() for nxt in neighbors(cur): ...</pre>

题号	题目	一句话识别
#200	岛屿数量	遍历网格，遇到陆地就 DFS/BFS flood fill 并标记访问。
#994	腐烂的橘子	多源 BFS：所有腐烂橘子先入队，逐层扩散并计时。
#207	课程表	建有向图和 indegree，Kahn BFS；最终处理节点数==课程数即无环。
#208	实现 Trie	节点包含 children 与 end 标记；insert/search/startsWith 沿字符逐层走。

8. 回溯

统一模板：选择列表 -> 做选择 -> 递归 -> 撤销选择；最重要的是 start/used/剪枝条件。



双语言核心模板

Go	Python
<pre>var dfs func(start int) dfs = func(start int) { if done() { saveCopy(path); return } for i := start; i < n; i++ { path = append(path, nums[i]) dfs(i+1) path = path[:len(path)-1] } }</pre>	<pre>def dfs(start): if done(): ans.append(path.copy()); return for i in range(start, n): path.append(nums[i]) dfs(i+1) path.pop()</pre>

题号	题目	一句话识别
#46	全排列	path + used；每层选择一个未使用元素，长度 n 时收集答案。
#78	子集	每个位置选/不选，或 startIndex 枚举下一项；每个节点都收集。
#17	电话号码的字母组合	按 digit 深度展开对应字母集合，走到末尾收集。
#39	组合总和	排序可剪枝；同一个 candidate 可重复选，因此递归仍传 i。
#22	括号生成	只在 open < n 时放 '(', close < open 时放 ')', 天然保证合法。
#79	单词搜索	从每格 DFS，匹配一个字符后标记访问，四方向继续，回溯恢复。
#131	分割回文串	枚举切分终点，仅当 s[start:end] 是回文才递归。
#51	N 皇后	逐行放皇后，用列、主对角线 r-c、副对角线 r+c 集合 O(1) 判冲突。

9. 二分搜索

不要死记题型，统一成“在单调谓词上找边界”。推荐先熟 lower_bound，再处理旋转数组的局部有序性。



双语言核心模板

Go	Python
<pre>l, r := 0, len(nums) // [l,r) for l < r { m := l + (r-l)/2 if nums[m] < target { l = m+1 } else { r = m } } return l</pre>	<pre>l, r = 0, len(nums) # [l,r) while l < r: m = (l+r)//2 if nums[m] < target: l = m+1 else: r = m return l</pre>

题号	题目	一句话识别
#35	搜索插入位置	lower_bound：找第一个 $\geq$ target 的位置。
#74	搜索二维矩阵	把 $m \times n$ 矩阵虚拟成一维有序数组，idx $\rightarrow$ row/col。
#34	在排序数组中查找元素的第一个和最后一个位置	分别求 lower_bound(target) 与 lower_bound(target+epsilon)/upper_bound(target)-1。
#33	搜索旋转排序数组	每轮至少有一半有序，判断 target 是否落在该有序区间决定舍弃哪半。
#153	寻找旋转排序数组中的最小值	比较 nums[mid] 与 nums[right]：mid 大则最小值在右侧，否则在含 mid 的左侧。
#4	寻找两个正序数组的中位数	在较短数组上二分分割点，使左右两侧元素数量相等且 $\max\text{Left} \leq \min\text{Right}$ 。

10. 栈 / 单调栈

普通栈处理嵌套与匹配；单调栈解决“下一个更大/更小”和“以当前元素为最值能扩到哪里”。



双语言核心模板

Go	Python
<pre>st := []int{} for i, x := range nums { for len(st)>0 && nums[st[len(st)-1]] < x { j := st[len(st)-1]; st = st[:len(st)-1] ans[j] = i-j } st = append(st, i) }</pre>	<pre>st = [] for i, x in enumerate(nums): while st and nums[st[-1]] < x: j = st.pop() ans[j] = i-j st.append(i)</pre>

题号	题目	一句话识别
#20	有效的括号	遇左括号入栈，右括号必须与栈顶匹配。
#155	最小栈	主栈每个元素同时存当前最小值，或额外 minStack。
#394	字符串解码	遇 '[' 保存当前字符串与 repeat，遇 ']' 弹出并拼接。
#739	每日温度	栈存未找到更高温度的下标并保持单调递减，新温度更大时持续弹栈结算距离。
#84	柱状图中最大的矩形	单调递增栈；遇更低高度时弹出柱子，以当前 i 为右边界计算最大矩形。

11. 堆

Top K 用大小 k 的堆；流式中位数用双堆平衡。Go 的 `heap.Interface` 和 Python `heapq` 都要熟。

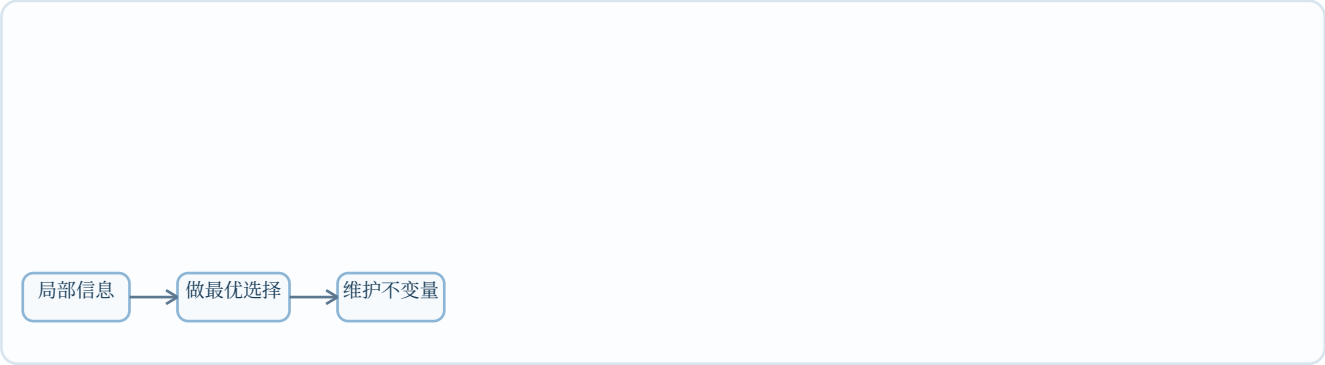


双语言核心模板

Go		Python
<pre>// container/heap: 小顶堆 Less/Push/Pop h := &MinHeap{} heap.Init(h) heap.Push(h, x) minv := heap.Pop(h).(int)</pre>		<pre>import heapq h = [] heapq.heappush(h, x) minv = heapq.heappop(h) # max-heap 用 -x</pre>
题号	题目	一句话识别
#215	数组中的第 K 个最大元素	维护大小 k 的小顶堆；或 QuickSelect 平均 O(n)。
#347	前 K 个高频元素	先计频，再用大小 k 的小顶堆或按频次桶排序。
#295	数据流的中位数	大顶堆存较小一半，小顶堆存较大一半，保持数量差≤1 且 <code>maxLow≤minHigh</code> 。

12. 贪心

贪心不是“看起来最优”，而是维护一个可证明的不变量或覆盖范围。



双语言核心模板

Go		Python
<pre>best := initial for i, x := range nums { // 初始化/更新 best = update(best, i, x) }</pre>		<pre>best = initial for i, x in enumerate(nums): # 初始化/更新 best = update(best, i, x)</pre>
题号	题目	一句话识别
#121	买卖股票的最佳时机	遍历维护历史最低价 minPrice，并更新 price-minPrice 最大值。
#55	跳跃游戏	维护当前能到达的最远位置 farthest；若 i>farthest 则失败。
#45	跳跃游戏 II	把一次跳跃看作一层区间；扫描当前层，维护下一层最远端，到 currentEnd 时步数+1。
#763	划分子母区间	先记录每字符最后位置，扫描时扩展当前段 end；i==end 时切段。

13. 动态规划

DP 四步：状态、选择/转移、初始化、遍历顺序。背包题再额外检查容量循环方向。



双语言核心模板

Go	Python
<pre>dp := make([]int, n+1) dp[0] = base for i := 1; i <= n; i++ { dp[i] = transition(dp, i) } return dp[n]</pre>	<pre>dp = [0] * (n+1) dp[0] = base for i in range(1, n+1): dp[i] = transition(dp, i) return dp[n]</pre>

题号	题目	一句话识别
#70	爬楼梯	$dp[i] = dp[i-1] + dp[i-2]$, 可滚动变量优化空间。
#118	杨辉三角	每行两端为 1, 中间等于上一行相邻两数之和。
#198	打家劫舍	$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ 。
#279	完全平方数	完全背包: $dp[i] = \min(dp[i-j^2]) + 1$ 。
#322	零钱兑换	完全背包最小值: 对 amount 从 coin 到 target 正向更新。
#139	单词拆分	$dp[i]$ 表示前 i 字符可拆; 枚举 $j < i$ 且 $dp[j]$ 且 $s[j:i]$ in dict。
#300	最长递增子序列	维护 tails: tails[len-1] 是该长度递增子序列的最小尾值, 用 lower_bound 替换。
#152	乘积最大子数组	同时维护 maxProd/minProd, 因为负数会交换最大最小。
#416	分割等和子集	总和为奇数直接 false; 目标 sum/2, 0/1 背包倒序更新可达性。
#32	最长有效括号	栈存最后一个无法匹配位置; 每次匹配成功长度 $= i - \text{stackTop}$ 。
#62	不同路径	$dp[i][j] = dp[i-1][j] + dp[i][j-1]$, 可压缩到一维。
#64	最小路径和	$dp[i][j] = \text{grid}[i][j] + \min(\text{top}, \text{left})$, 可原地或一维压缩。
#5	最长回文子串	中心扩展最简: 每个位置尝试奇/偶中心并更新最长。
#1143	最长公共子序列	字符相等 $dp[i][j] = dp[i-1][j-1] + 1$, 否则 $\max(\text{top}, \text{left})$ 。
#72	编辑距离	$dp[i][j]$ 表示 word1 前 i 到 word2 前 j 的最少操作; 取插入/删除/替换最小。

14. 高频技巧

位运算、投票、三路分区、字典序、Floyd 都属于“面试一眼识别”的技巧题，适合最后阶段集中记忆。



双语言核心模板

Go	Python
<pre>// Go: hash lookup m := map[int]int{} for i, x := range nums { if j, ok := m[target-x]; ok { return []int{j,i} } m[x] = i }</pre>	<pre># Python: hash lookup pos = {} for i, x in enumerate(nums): if target-x in pos: return [pos[target-x], i] pos[x] = i</pre>

题号	题目	一句话识别
#136	只出现一次的数字	所有数异或，成对数字抵消。
#169	多数元素	Boyer-Moore：相同票数+1，不同-1，归零换候选。
#75	颜色分类	Dutch National Flag：0 放左、2 放右、1 留中间。
#31	下一个排列	从右找首个下降位置 i；再找右侧刚好更大的 j 交换；最后反转后缀。
#287	寻找重复数	把 nums[i] 看成 next 指针，问题变成环入口；用 Floyd 找入口。

三、15 道母题：必须能双语言默写

这些题覆盖大多数可迁移模式。第一轮允许看模板；第二轮应做到 15-25 分钟内独立写出，并用另一语言复写核心逻辑。

#1 两数之和 / Two Sum

为什么是母题：最小哈希母题：训练“先查再写”和 Go/Python map 语法。



核心思路：遍历时查 target-x 是否已出现；先查后放避免同一元素重复使用。

复杂度：O(n) / O(n) 易错：value->index 覆盖无妨，但顺序必须先查再写。扩展：扩展到 Two Sum II、3Sum、kSum。

Go	Python
<pre>func twoSum(nums []int, target int) []int { pos := map[int]int{} for i, x := range nums { if j, ok := pos[target-x]; ok { return []int{j, i} } pos[x] = i } return nil }</pre>	<pre>def twoSum(nums, target): pos = {} for i, x in enumerate(nums): if target - x in pos: return [pos[target-x], i] pos[x] = i</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#15 三数之和 / 3Sum

为什么是母题：排序 + 双指针 + 去重，是 kSum 系列入口。



核心思路：排序后固定 i，对剩余区间双指针；严格跳过重复值。

复杂度： $O(n^2)$ / $O(1)$ 易错：去重位置：i、left、right 都要正确处理。扩展：推广 kSum、4Sum，递归降维。

Go	Python
<pre>func threeSum(nums []int) [][]int { sort.Ints(nums); ans := [][]int{} for i:=0; i<len(nums)-2; i++ { if i>0 && nums[i]==nums[i-1] { continue } l,r := i+1,len(nums)-1 for l<r { s:=nums[i]+nums[l]+nums[r] if s<0 { l++ } else if s>0 { r-- } else { ans=append(ans, []int{nums[i],nums[l],nums[r]}) l++; r-- for l<r && nums[l]==nums[l-1] { l++ } for l<r && nums[r]==nums[r+1] { r-- } } } } return ans }</pre>	<pre>def threeSum(nums): nums.sort(); ans=[] for i in range(len(nums)-2): if i and nums[i]==nums[i-1]: continue l,r=i+1,len(nums)-1 while l<r: s=nums[i]+nums[l]+nums[r] if s<0: l+=1 elif s>0: r-=1 else: ans.append([nums[i],nums[l],nums[r]]) l+=1; r-=1 while l<r and nums[l]==nums[l-1]: l+=1 while l<r and nums[r]==nums[r+1]: r-=1 return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#42 接雨水 / Trapping Rain Water

为什么是母题：理解双指针为何正确，比记代码更重要；也要会单调栈版本。



核心思路：双指针维护 leftMax/rightMax；较小一侧的水量已由该侧最大值决定。

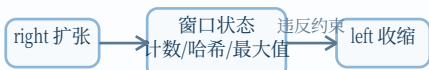
复杂度：O(n) / O(1) 易错：更新 max 与计算水量的顺序。扩展：可用前后缀数组或单调栈。

Go	Python
<pre>func trap(h []int) int { l, r, ans := 0, len(h)-1, 0 lm, rm := 0, 0 for l < r { if h[l] <= h[r] { lm = max(lm, h[l]); ans += lm - h[l]; l++ } else { rm = max(rm, h[r]); ans += rm - h[r]; r-- } } return ans }</pre>	<pre>def trap(h): l, r, ans = 0, len(h)-1, 0 lm = rm = 0 while l < r: if h[l] <= h[r]: lm = max(lm, h[l]); ans += lm - h[l]; l += 1 else: rm = max(rm, h[r]); ans += rm - h[r]; r -= 1 return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#3 无重复字符的最长子串 / Longest Substring Without Repeating Characters

为什么是母题：可变长度窗口模板，left 不回退。



核心思路：right 扩张，记录字符最近位置或计数；重复时移动 left。

复杂度：O(n) / O(字符集) 易错：left 只能右移，不能回退。扩展：模板：最多 K 种字符、替换后最长重复字符。

Go	Python
<pre>func lengthOfLongestSubstring(s string) int { last:=map[byte]int{}; left,ans:=0,0 for r:=0;r<len(s);r++ { if p,ok:=last[s[r]]; ok && p>=left { left=p+1 } last[s[r]]=r; ans=max(ans,r-left+1) } return ans }</pre>	<pre>def lengthOfLongestSubstring(s): last={}; left=ans=0 for r,ch in enumerate(s): if ch in last and last[ch]>=left: left=last[ch]+1 last[ch]=r ans=max(ans,r-left+1) return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#560 和为 K 的子数组 / Subarray Sum Equals K

为什么是母题：前缀和 + 计数哈希，树路径和也会复用。



核心思路： $\text{prefix}[j] - \text{prefix}[i] = k$ ，边扫边统计 $\text{prefix} - k$ 出现次数。

复杂度： $O(n)$ / $O(n)$ 易错：初始化 $\text{cnt}[0] = 1$ 是关键。扩展：二维前缀和、和可被 K 整除。

Go	Python
<pre>func subarraySum(nums []int, k int) int { cnt:=map[int]int{0:1}; pre,ans:=0,0 for _,x:=range nums { pre+=x; ans+=cnt[pre-k]; cnt[pre]++ } return ans }</pre>	<pre>def subarraySum(nums, k): cnt={0:1}; pre=ans=0 for x in nums: pre+=x ans+=cnt.get(pre-k,0) cnt[pre]=cnt.get(pre,0)+1 return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#206 反转链表 / Reverse Linked List

为什么是母题：所有链表原地重连题的基础。



核心思路：prev/cur/next 三指针逐个反转 next。

复杂度：O(n) / O(1) 易错：一定先保存 next。扩展：递归反转、区间反转。

Go	Python
<pre>func reverseList(head *ListNode) *ListNode { var prev *ListNode cur:=head for cur!=nil { nxt:=cur.Next; cur.Next=prev prev,cur=cur,nxt } return prev }</pre>	<pre>def reverseList(head): prev,cur=None,head while cur: nxt=cur.next cur.next=prev prev,cur=cur,nxt return prev</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#146 LRU 缓存 / LRU Cache

为什么是母题：面试常考的数据结构设计题：哈希 + 双向链表。



核心思路：哈希定位节点 + 双向链表维护最近使用顺序；get/put 都 $O(1)$ 。

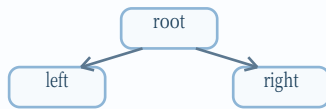
复杂度： $O(1)$ / $O(\text{capacity})$ 易错：更新现有 key 也要移动到头部；淘汰尾部。扩展：LFU、TTL cache、并发安全缓存。

Go	Python
<pre>type node struct { key,val int; prev,next *node } type LRUCache struct { cap int; m map[int]*node; head,tail *node } // get: map 查找 -> remove -> addFront // put: 查找 head/tail 位置 O(1) // 更新现有 key 也要移动到头部</pre>	<pre>class Node: def __init__(self,k=0,v=0): self.k,self.v=k,v; self.prev=self.next=None class LRUCache: # dict key->Node + doubly linked list # get: move_to_front # put: update/insert; evict tail.prev when over capacity pass</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#102 二叉树的层序遍历 / Binary Tree Level Order Traversal

为什么是母题：BFS 层序模板，右视图/最短层数等题直接复用。



核心思路：BFS，每层先读取 queue 当前长度。

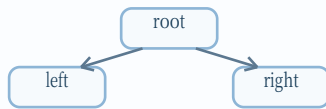
复杂度：O(n) / O(w) 易错：层边界用 size 固定。扩展：右视图、锯齿层序。

Go	Python
<pre>func levelOrder(root *TreeNode) [][]int { if root==nil { return nil } q:=[]*TreeNode{root}; ans=[][]int{} for len(q)>0 { n:=len(q); row:=[]int{} for i:=0;i<n;i++ { x:=q[0]; q=q[1:]; row=append(row,x.Val) if x.Left!=nil { q=append(q,x.Left) } if x.Right!=nil { q=append(q,x.Right) } } ans=append(ans,row) } return ans }</pre>	<pre>def levelOrder(root): if not root: return [] from collections import deque q=deque([root]); ans=[] while q: row=[] for _ in range(len(q)): x=q.popleft(); row.append(x.val) if x.left: q.append(x.left) if x.right: q.append(x.right) ans.append(row) return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#124 二叉树中的最大路径和 / Binary Tree Maximum Path Sum

为什么是母题：典型树 DP：返回值与全局答案的语义不同。



核心思路：后序返回“向上最大贡献”，负贡献截断为 0；全局答案考虑左右贡献+root。

复杂度：O(n) / O(h) 易错：全负数时全局 ans 初始化负无穷。扩展：树 DP 路径母题。

Go	Python
<pre>func maxPathSum(root *TreeNode) int { ans:=math.MinInt var dfs func(*TreeNode) int dfs=func(n *TreeNode) int { if n==nil { return 0 } l:=max(0,dfs(n.Left)); r:=max(0,dfs(n.Right)) ans=max(ans,n.Val+l+r) return n.Val+max(l,r) } dfs(root); return ans }</pre>	<pre>def maxPathSum(root): ans=float('-inf') def dfs(n): nonlocal ans if not n: return 0 l=max(0,dfs(n.left)); r=max(0,dfs(n.right)) ans=max(ans,n.val+l+r) return n.val+max(l,r) dfs(root); return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#200 岛屿数量 / Number of Islands

为什么是母题：网格 DFS/BFS 的基础模板。



核心思路：遍历网格，遇到陆地就 DFS/BFS flood fill 并标记访问。

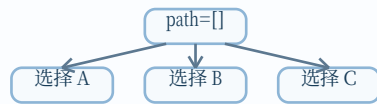
复杂度：O(mn) / O(mn) 易错：原地改 grid 可省 visited。扩展：岛屿面积、周长、并查集。

Go	Python
<pre>func numIslands(g [][]byte) int { m,n:=len(g),len(g[0]); ans:=0 var dfs func(int,int) dfs=func(r,c int){ if r<0 r>=m c<0 c>=n g[r][c]!='1' { return } g[r][c]='0' dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1) } for r:=0;r<m;r++ { for c:=0;c<n;c++ { if g[r][c]=='1' { ans++; dfs(r,c) } }} return ans }</pre>	<pre>def numIslands(g): m,n=len(g),len(g[0]); ans=0 def dfs(r,c): if not (0<=r<m and 0<=c<n and g[r][c]!='1'): return g[r][c]='0' for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)): dfs(r+dr,c+dc) for r in range(m): for c in range(n): if g[r][c]=='1': ans+=1; dfs(r,c) return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#46 全排列 / Permutations

为什么是母题：回溯的最标准 used/path 模板。



核心思路：path + used；每层选择一个未使用元素，长度 n 时收集答案。

复杂度： $O(n \cdot n!)$ / $O(n)$ 易错：结果要复制 path。扩展：含重复元素的 Permutations II。

Go	Python
<pre>func permute(nums []int) [][]int { ans:=[][]int{}; path:=[]int{}; used:=make([]bool,len(nums)) var dfs func() dfs=func(){ if len(path)==len(nums) { ans=append(ans, append([]int(nil),path...)); return } for i,x:=range nums { if !used[i] { used[i]=true; path=append(path,x); dfs() path=path[:len(path)-1]; used[i]=false }} }; dfs(); return ans }</pre>	<pre>def permute(nums): ans=[]; path=[]; used=[False]*len(nums) def dfs(): if len(path)==len(nums): ans.append(path.copy()); return for i,x in enumerate(nums): if not used[i]: used[i]=True; path.append(x); dfs() path.pop(); used[i]=False dfs(); return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#33 搜索旋转排序数组 / Search in Rotated Sorted Array

为什么是母题：旋转数组二分，考“局部单调性”。



核心思路：每轮至少有一半有序，判断 target 是否落在该有序区间决定舍弃哪半。

复杂度： $O(\log n)$ / $O(1)$ 易错：等号边界决定哪半有序。扩展：含重复元素版本。

Go	Python
<pre>func search(nums []int, target int) int { l,r:=0,len(nums)-1 for l<=r { m:=(l+r)/2 if nums[m]==target { return m } if nums[l]<=nums[m] { if nums[l]<=target && target<nums[m] { r=m-1 } else { l=m+1 } } else { if nums[m]<target && target<=nums[r] { l=m+1 } else { r=m-1 } } } return -1 }</pre>	<pre>def search(nums, target): l,r=0,len(nums)-1 while l<=r: m=(l+r)//2 if nums[m]==target: return m if nums[l]<=nums[m]: if nums[l]<=target<nums[m]: r=m-1 else: l=m+1 else: if nums[m]<target<=nums[r]: l=m+1 else: r=m-1 return -1</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#739 每日温度 / Daily Temperatures

为什么是母题：单调栈的母题。



核心思路：栈存未找到更高温度的下标并保持单调递减，新温度更大时持续弹栈结算距离。

复杂度： $O(n)$ / $O(n)$ 易错：答案是下标差。扩展：Next Greater Element 系列。

Go	Python
<pre>func dailyTemperatures(t []int) []int { ans:=make([]int,len(t)); st:=[]int{} for i,x:=range t { for len(st)>0 && t[st[len(st)-1]]<x { j:=st[len(st)-1]; st=st[:len(st)-1]; ans[j]=i-j } st=append(st,i) } return ans }</pre>	<pre>def dailyTemperatures(t): ans=[0]*len(t); st=[] for i,x in enumerate(t): while st and t[st[-1]]<x: j=st.pop(); ans[j]=i-j st.append(i) return ans</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#295 数据流的中位数 / Find Median from Data Stream

为什么是母题：双堆平衡，训练 heap 方向和不变量。



核心思路：大顶堆存较小一半，小顶堆存较大一半，保持数量差 ≤ 1 且 $\max\text{Low} \leq \min\text{High}$ 。

复杂度：add $O(\log n)$, find $O(1)$ 易错：Go 大顶堆可存负数简化。扩展：滑动窗口中位数需延迟删除。

Go	Python
<pre>// small: 小顶堆, large: 大顶堆 // addNum(x): 保持 size 差 <= 1 // findMedian(): size 差 <= 1 // Go 实现 heap.Interface 接口</pre>	<pre>import heapq class MedianFinder: def __init__(self): self.small=[]; self.large=[] def addNum(self,x): heapq.heappush(self.small,-x) heapq.heappush(self.large,-heapq.heappop(self.small)) if len(self.large)>len(self.small): heapq.heappush(self.small,-heapq.heappop(self.large)) def findMedian(self): if len(self.small)>len(self.large): return -self.small[0] return (-self.small[0]+self.large[0])/2</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

#322 零钱兑换 / Coin Change

为什么是母题：完全背包与循环方向的高频考点。



核心思路：完全背包最小值：对 amount 从 coin 到 target 正向更新。

复杂度： $O(\text{amount} * \text{coins}) / O(\text{amount})$ 易错：无解返回 -1；循环方向与 0/1 背包不同。扩展：组合数 Coin Change II。

Go	Python
<pre>func coinChange(coins []int, amount int) int { inf:=amount+1; dp:=make([]int,amount+1) for i:=1;i<=amount;i++ { dp[i]=inf } for _,c:=range coins { for x:=c;x<=amount;x++ { dp[x]=min(dp[x],dp[x-c]+1) } } if dp[amount]==inf { return -1 }; return dp[amount] }</pre>	<pre>def coinChange(coins, amount): inf=amount+1; dp=[0]+[inf]*amount for c in coins: for x in range(c, amount+1): dp[x]=min(dp[x], dp[x-c]+1) return -1 if dp[amount]==inf else dp[amount]</pre>

复盘问题：不用看代码，能否先画出状态变化？能否用一句话说出不变量？如果把输入规模放大 10 倍，瓶颈在哪里？

四、Hot 100 逐题速览

这部分用于第一轮当天复盘和第二轮查漏补缺。每题至少要能在 60 秒内讲出“模式 + 核心不变量 + 复杂度 + 易错点”。

#1 两数之和 [Easy] Two Sum
模式：哈希 思路：遍历时查 target-x 是否已出现；先查后放避免同一元素重复使用。 复杂度： $O(n)$ / $O(n)$ 易错：value->index 覆盖无妨，但顺序必须先查再写。 扩展：扩展到 Two Sum II、3Sum、kSum。 双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。
#49 字母异位词分组 [Medium] Group Anagrams
模式：哈希 思路：把每个字符串映射到稳定签名：排序串或 26 字母计数 tuple/array。 复杂度： $O(n*k \log k)$ 或 $O(n*k)$ 易错：Go 的数组可作为 map key；Python list 不可哈希，要 tuple。 扩展：字符集扩大时用排序签名或稀疏计数。 双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。
#128 最长连续序列 [Medium] Longest Consecutive Sequence
模式：哈希 思路：把数放 set，只从没有前驱 x-1 的数开始向右扩展。 复杂度： $O(n)$ / $O(n)$ 易错：不要从每个点都扩展，否则退化 $O(n^2)$ 。 扩展：可延伸到区间合并、并查集解法。 双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。
#283 移动零 [Easy] Move Zeroes
模式：双指针 思路：slow 指向下一个非零写入位置，fast 扫描；可交换或覆盖后补零。 复杂度： $O(n)$ / $O(1)$ 易错：保持非零相对顺序。 扩展：同类：移除元素、颜色分类。 双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#11 盛最多水的容器 [Medium] Container With Most Water

模式：双指针
思路：左右夹逼，面积由短板决定，因此移动短板才可能变好。
复杂度： $O(n) / O(1)$
易错：移动长板没有收益证明要讲清。
扩展：双指针不变量与贪心证明。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#15 三数之和 [Medium] 3Sum

模式：双指针
思路：排序后固定 i，对剩余区间双指针；严格跳过重复值。
复杂度： $O(n^2) / O(1)$
易错：去重位置：i、left、right 都要正确处理。
扩展：推广 kSum、4Sum，递归降维。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#42 接雨水 [Hard] Trapping Rain Water

模式：双指针
思路：双指针维护 leftMax/rightMax；较小一侧的水量已由该侧最大值决定。
复杂度： $O(n) / O(1)$
易错：更新 max 与计算水量的顺序。
扩展：可用前后缀数组或单调栈。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#3 无重复字符的最长子串 [Medium] Longest Substring Without Repeating Characters

模式：滑动窗口
思路：right 扩张，记录字符最近位置或计数；重复时移动 left。
复杂度： $O(n) / O(\text{字符集})$
易错：left 只能右移，不能回退。
扩展：模板：最多 K 种字符、替换后最长重复字符。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#438 找到字符串中所有字母异位词 [Medium] Find All Anagrams in a String

模式：滑动窗口
思路：固定长度窗口维护频次差；窗口达到 len(p) 后判断是否匹配。
复杂度： $O(n) / O(\text{字符集})$
易错：不要每次复制/排序整个窗口。
扩展：最小覆盖子串、排列包含。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#560 和为 K 的子数组 [Medium] Subarray Sum Equals K

模式：前缀和
思路： $\text{prefix}[j] - \text{prefix}[i] = k$ ，边扫边统计 prefix-k 出现次数。
复杂度： $O(n) / O(n)$
易错：初始化 $\text{cnt}[0] = 1$ 是关键。
扩展：二维前缀和、和可被 K 整除。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#239 滑动窗口最大值 [Hard] Sliding Window Maximum

模式：单调队列
思路：双端队列存下标，保持对应值单调递减；过期下标从队头弹出。
复杂度： $O(n) / O(k)$
易错：队列存下标不是值。
扩展：最小值窗口、单调队列 DP。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#76 最小覆盖子串 [Hard] Minimum Window Substring

模式：滑动窗口
思路：窗口扩张直到满足 need，再收缩 left 寻找最短合法窗口。
复杂度： $O(n) / O(\text{字符集})$
易错：valid 的定义与重复字符计数。
扩展：所有“最短满足条件窗口”的统一模板。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#53 最大子数组和 [Medium] Maximum Subarray

模式：数组/DP
思路：Kadane：以当前位置结尾的最优值 = $\max(\text{nums}[i], \text{prev} + \text{nums}[i])$ 。
复杂度： $O(n) / O(1)$
易错：全负数时不能初始化为 0。
扩展：返回区间端点、最大环形子数组。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#56 合并区间 [Medium] Merge Intervals

模式：数组
思路：按 start 排序，若当前 $\text{start} \leq \text{last.end}$ 就合并，否则新建区间。
复杂度： $O(n \log n) / O(n)$
易错：边界是否重叠要按题意处理。
扩展：插入区间、会议室、扫描线。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#189 轮转数组 [Medium] Rotate Array

模式：数组
思路：三次反转：整体反转，再反转前 k 和后 $n-k$ 。
复杂度： $O(n) / O(1)$
易错： $k \% n$ ；空数组防除零。
扩展：环状替换/最大公约数分组。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#238 除自身以外数组的乘积 [Medium] Product of Array Except Self

模式：数组
思路：答案先存前缀积，再反向乘后缀积，避免除法。
复杂度： $O(n) / O(1)$ 额外
易错：0 的处理自然包含在前后缀积里。
扩展：前缀/后缀积通用技巧。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#41 缺失的第一个正数 [Hard] First Missing Positive

模式：数组
思路：把值 x 放到下标 $x-1$ 的位置，原地哈希；最后找首个 $\text{nums}[i] \neq i+1$ 。
复杂度： $O(n) / O(1)$
易错：交换必须 while，且防重复值死循环。
扩展：原地桶定位类题。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#73 矩阵置零 [Medium] Set Matrix Zeroes

模式：矩阵
思路：用首行首列充当 marker，并单独记录首行/首列是否原本含 0。
复杂度： $O(mn) / O(1)$
易错：处理首行首列顺序。
扩展：原地标记技巧。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#54 螺旋矩阵 [Medium] Spiral Matrix

模式：矩阵
思路：维护 top/bottom/left/right 四边界，逐层收缩。
复杂度： $O(mn) / O(1)$
易错：单行/单列时避免重复遍历。
扩展：矩阵层序模拟。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#48 旋转图像 [Medium] Rotate Image

模式：矩阵
思路：转置 + 每行反转可得到顺时针 90 度旋转。
复杂度： $O(n^2) / O(1)$
易错：必须原地操作。
扩展：逆时针、180 度旋转。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#240 搜索二维矩阵 II [Medium] Search a 2D Matrix II

模式：矩阵
思路：从右上角开始：目标小则左移，大则下移。
复杂度： $O(m+n) / O(1)$
易错：利用行列双调性。
扩展：Young tableau 类搜索。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#160 相交链表 [Easy] Intersection of Two Linked Lists

模式：链表
思路：两个指针走完自己的链再走对方链，路径长度对齐后在交点相遇。
复杂度： $O(m+n) / O(1)$
易错：比较节点地址，不是值。
扩展：链表长度差法。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#206 反转链表 [Easy] Reverse Linked List

模式：链表
思路：prev/cur/next 三指针逐个反转 next。
复杂度： $O(n) / O(1)$
易错：一定先保存 next。
扩展：递归反转、区间反转。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#234 回文链表 [Easy] Palindrome Linked List

模式：链表
思路：快慢指针找中点，反转后半段，再双指针比较。
复杂度： $O(n) / O(1)$
易错：奇偶长度与是否恢复链表。
扩展：链表原地操作组合题。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#141 环形链表 [Easy] Linked List Cycle

模式：链表
思路：快慢指针；若有环最终相遇。
复杂度： $O(n) / O(1)$
易错：fast 与 fast.next 判空。
扩展：为 Cycle II 做铺垫。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#142 环形链表 II [Medium] Linked List Cycle II

模式：链表
思路：快慢相遇后，一指针回头部，两者同速再次相遇即环入口。
复杂度： $O(n) / O(1)$
易错：需能解释数学关系。
扩展：环长度、重复数 Floyd 解法。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#21 合并两个有序链表 [Easy] Merge Two Sorted Lists

模式：链表
思路：dummy + tail 每次接较小节点。
复杂度： $O(m+n) / O(1)$
易错：最后接剩余链。
扩展：Merge k List 的基础。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#2 两数相加 [Medium] Add Two Numbers

模式：链表
思路：同步遍历两链表并维护 carry，直到两链与 carry 都为空。
复杂度： $O(\max(m,n)) / O(1)$
易错：最后一位进位。
扩展：大整数链表运算。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#19 删除链表倒数第 N 个结点 [Medium] Remove Nth Node From End of List

模式：链表
思路：dummy；fast 先走 n 步，再 fast/slow 同步直到 fast 到尾。
复杂度： $O(n) / O(1)$
易错：删除头节点需要 dummy。
扩展：固定距离双指针。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#24 两两交换链表中的节点 [Medium] Swap Nodes in Pairs

模式：链表
思路：dummy 前驱 pre，每轮重连 a,b 三条边。
复杂度： $O(n) / O(1)$
易错：注意保存 b.next。
扩展：k 组反转的简化版。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#25 K 个一组翻转链表 [Hard] Reverse Nodes in k-Group

模式：链表
思路：每次先确认剩余至少 k 个，再反转 [groupHead, groupTail] 并拼接。
复杂度： $O(n) / O(1)$
易错：不足 k 个保持原样。
扩展：区间反转模板。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#138 随机链表的复制 [Medium] Copy List with Random Pointer

模式：链表
思路：哈希 old->new 两遍建立；或原节点后插副本再拆分。
复杂度： $O(n) / O(n)$ 或 $O(1)$
易错：random 可能为空。
扩展：复杂结构深拷贝。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#148 排序链表 [Medium] Sort List

模式：链表
思路：归并排序：快慢分割，递归排序左右，再 merge。
复杂度： $O(n \log n) / O(\log n)$
易错：断开 mid 前的链。
扩展：自底向上归并可 $O(1)$ 栈。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#23 合并 K 个升序链表 [Hard] Merge k Sorted Lists

模式：链表/堆
思路：小顶堆维护每条链当前头节点；弹出最小并推进对应链。
复杂度： $O(N \log k) / O(k)$
易错：Go heap.Interface 实现细节。
扩展：分治两两 merge 也是 $O(N \log k)$ 。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#146 LRU 缓存 [Medium] LRU Cache

模式：链表/哈希
思路：哈希定位节点 + 双向链表维护最近使用顺序；get/put 都 $O(1)$ 。
复杂度： $O(1) / O(\text{capacity})$
易错：更新现有 key 也要移动到头部；淘汰尾部。
扩展：LFU、TTL cache、并发安全缓存。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#94 二叉树的中序遍历 [Easy] Binary Tree Inorder Traversal

模式：二叉树
思路：递归 left-root-right；迭代用栈一路压左。
复杂度： $O(n) / O(h)$
易错：迭代模板要熟练。
扩展：前序/后序/Morris 遍历。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#104 二叉树的最大深度 [Easy] Maximum Depth of Binary Tree

模式：二叉树
思路：DFS 返回 $1 + \max(\text{left}, \text{right})$ ，或 BFS 层数。
复杂度： $O(n) / O(h)$
易错：空树深度为 0。
扩展：最小深度、树高。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#226 翻转二叉树 [Easy] Invert Binary Tree

模式：二叉树
思路：递归交换左右子树。
复杂度： $O(n) / O(h)$
易错：交换可在递归前后。
扩展：镜像/对称树。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#101 对称二叉树 [Easy] Symmetric Tree

模式：二叉树
思路：递归比较 left.left vs right.right 、 left.right vs right.left 。
复杂度： $O(n) / O(h)$
易错：两个子树必须镜像比较。
扩展：树同构。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#543 二叉树的直径 [Easy] Diameter of Binary Tree

模式：二叉树
思路：后序 DFS 返回高度，同时用 $\text{leftHeight} + \text{rightHeight}$ 更新答案。
复杂度： $O(n) / O(h)$
易错：直径按边数还是节点数。
扩展：树 DP：路径问题。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#102 二叉树的层序遍历 [Medium] Binary Tree Level Order Traversal

模式：二叉树
思路：BFS，每层先读取 queue 当前长度。
复杂度： $O(n) / O(w)$
易错：层边界用 size 固定。
扩展：右视图、锯齿层序。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#108 将有序数组转换为二叉搜索树 [Easy] Convert Sorted Array to BST

模式：二叉树
思路：取中点为根，递归构造左右区间。
复杂度： $O(n) / O(\log n)$
易错：边界统一用闭区间或半开区间。
扩展：平衡 BST 构建。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#98 验证二叉搜索树 [Medium] Validate Binary Search Tree

模式：二叉树
思路：DFS 传递合法 (low,high)；或中序必须严格递增。
复杂度： $O(n) / O(h)$
易错：不能只比较父子；重复值不合法。
扩展：BST 通用上下界约束。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#230 二叉搜索树中第 K 小的元素 [Medium] Kth Smallest Element in a BST

模式：二叉树
思路：BST 中序有序，遍历到第 k 个即可。
复杂度： $O(h+k) / O(h)$
易错：k 是 1-based。
扩展：增强 BST 维护 subtree size。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#199 二叉树的右视图 [Medium] Binary Tree Right Side View

模式：二叉树

思路：BFS 每层取最后一个；或 DFS 先右后左按深度首次访问。

复杂度： $O(n) / O(h \text{ or } w)$

易错：DFS 版本的“首次到达深度”技巧。

扩展：左视图、边界视图。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#114 二叉树展开为链表 [Medium] Flatten Binary Tree to Linked List

模式：二叉树

思路：后序把左子树插到右侧，再把原右子树接到左子树最右节点。

复杂度： $O(n) / O(h)$

易错：最终全部 left=nil。

扩展：原地树结构改写。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#105 从前序与中序遍历序列构造二叉树 [Medium] Construct Binary Tree from Preorder and Inorder Traversal

模式：二叉树

思路：前序首元素是根，用 inorder 哈希定位左右子树规模递归切分。

复杂度： $O(n) / O(n)$

易错：索引边界最易错。

扩展：中序+后序重建。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#437 路径总和 III [Medium] Path Sum III

模式：二叉树/前缀和

思路：DFS 路径前缀和 + 哈希统计 prefix-target；回溯时减计数。

复杂度： $O(n) / O(h)$

易错：离开节点必须回滚哈希。

扩展：树上的前缀和。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#236 二叉树的最近公共祖先 [Medium] Lowest Common Ancestor of a Binary Tree

模式：二叉树
思路：若 root 是 p/q 或为空直接返；左右递归都非空则 root 为 LCA。
复杂度： $O(n) / O(h)$
易错：普通树与 BST 的 LCA 区别。
扩展：倍增 LCA（多查询）。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#124 二叉树中的最大路径和 [Hard] Binary Tree Maximum Path Sum

模式：二叉树
思路：后序返回“向上最大贡献”，负贡献截断为 0；全局答案考虑左右贡献+root。
复杂度： $O(n) / O(h)$
易错：全负数时全局 ans 初始化负无穷。
扩展：树 DP 路径母题。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#200 岛屿数量 [Medium] Number of Islands

模式：图
思路：遍历网格，遇到陆地就 DFS/BFS flood fill 并标记访问。
复杂度： $O(mn) / O(mn)$
易错：原地改 grid 可省 visited。
扩展：岛屿面积、周长、并查集。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#994 腐烂的橘子 [Medium] Rotting Oranges

模式：图/BFS
思路：多源 BFS：所有腐烂橘子先入队，逐层扩散并计时。
复杂度： $O(mn) / O(mn)$
易错：最终检查剩余新鲜数。
扩展：最短传播时间、多源 BFS。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#207 课程表 [Medium] Course Schedule

模式：图/拓扑

思路：建有向图和 indegree, Kahn BFS；最终处理节点数==课程数即无环。

复杂度： $O(V+E) / O(V+E)$

易错：边方向 prereq -> course。

扩展：Course Schedule II 输出拓扑序。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#208 实现 Trie [Medium] Implement Trie

模式：Trie

思路：节点包含 children 与 end 标记；insert/search/startsWith 沿字符逐层走。

复杂度： $O(L) / O(\text{总字符数})$

易错：search 必须检查 end。

扩展：前缀统计、词频、AC 自动机。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#46 全排列 [Medium] Permutations

模式：回溯

思路：path + used；每层选择一个未使用元素，长度 n 时收集答案。

复杂度： $O(n*n!) / O(n)$

易错：结果要复制 path。

扩展：含重复元素的 Permutations II。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#78 子集 [Medium] Subsets

模式：回溯

思路：每个位置选/不选，或 startIndex 枚举下一项；每个节点都收集。

复杂度： $O(n*2^n) / O(n)$

易错：子集题不是只在叶子收集。

扩展：子集 II、位运算枚举。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#17 电话号码的字母组合 [Medium] Letter Combinations of a Phone Number

模式：回溯
思路：按 digit 深度展开对应字母集合，走到末尾收集。
复杂度： $O(4^n) / O(n)$
易错：空输入返回空列表。
扩展：笛卡尔积类回溯。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#39 组合总和 [Medium] Combination Sum

模式：回溯
思路：排序可剪枝；同一个 candidate 可重复选，因此递归仍传 i。
复杂度：指数级
易错：startIndex 决定组合不重复。
扩展：Combination Sum II/III。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#22 括号生成 [Medium] Generate Parentheses

模式：回溯
思路：只在 open < n 时放 '('，close < open 时放 ')'，天然保证合法。
复杂度：Catalan 级
易错：不要生成后再验证。
扩展：括号 DP/Catalan 数。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#79 单词搜索 [Medium] Word Search

模式：回溯
思路：从每格 DFS，匹配一个字符后标记访问，四方向继续，回溯恢复。
复杂度： $O(mn \cdot 4^L)$
易错：访问标记必须恢复。
扩展：Trie + 多词搜索 Word Search II。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#131 分割回文串 [Medium] Palindrome Partitioning

模式：回溯/DP
思路：枚举切分终点，仅当 $s[start:end]$ 是回文才递归。
复杂度：指数级
易错：可预计算 palindrome DP 降低重复判断。
扩展：最少回文切割。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#51 N 皇后 [Hard] N-Queens

模式：回溯
思路：逐行放皇后，用列、主对角线 $r-c$ 、副对角线 $r+c$ 集合 $O(1)$ 判冲突。
复杂度： $O(n!)$
易错：对角线编码是高频点。
扩展：N-Queens II 只计数，位运算优化。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#35 搜索插入位置 [Easy] Search Insert Position

模式：二分
思路：lower_bound：找第一个 $\geq target$ 的位置。
复杂度： $O(\log n) / O(1)$
易错：推荐 $[l, r)$ 模板减少边界错误。
扩展：lower/upper bound 母题。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#74 搜索二维矩阵 [Medium] Search a 2D Matrix

模式：二分
思路：把 $m*n$ 矩阵虚拟成一维有序数组， $idx \rightarrow row/col$ 。
复杂度： $O(\log mn) / O(1)$
易错：与 240 不同：这里整体有序。
扩展：二维到一维映射。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#34 在排序数组中查找元素的第一个和最后一个位置 [Medium] Find First and Last Position

模式：二分
思路：分别求 $\text{lower_bound}(\text{target})$ 与 $\text{lower_bound}(\text{target}+\text{epsilon})/\text{upper_bound}(\text{target})-1$ 。
复杂度： $O(\log n) / O(1)$
易错：不存在 target 时返回 $[-1,-1]$ 。
扩展：边界二分。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#33 搜索旋转排序数组 [Medium] Search in Rotated Sorted Array

模式：二分
思路：每轮至少有一半有序，判断 target 是否落在该有序区间决定舍弃哪半。
复杂度： $O(\log n) / O(1)$
易错：等号边界决定哪半有序。
扩展：含重复元素版本。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#153 寻找旋转排序数组中的最小值 [Medium] Find Minimum in Rotated Sorted Array

模式：二分
思路：比较 $\text{nums}[\text{mid}]$ 与 $\text{nums}[\text{right}]$ ：mid 大则最小值在右侧，否则在含 mid 的左侧。
复杂度： $O(\log n) / O(1)$
易错： $\text{right}=\text{mid}$ 不能 $\text{mid}-1$ 。
扩展：含重复值时退化处理。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#4 寻找两个正序数组的中位数 [Hard] Median of Two Sorted Arrays

模式：二分
思路：在较短数组上二分分割点，使左右两侧元素数量相等且 $\text{maxLeft} \leq \text{minRight}$ 。
复杂度： $O(\log \min(m,n))$
易错：哨兵 $\pm \text{inf}$ 与奇偶长度。
扩展：第 k 小元素的二分。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#20 有效的括号 [Easy] Valid Parentheses

模式：栈
思路：遇左括号入栈，右括号必须与栈顶匹配。
复杂度： $O(n) / O(n)$
易错：空栈访问与最终栈非空。
扩展：表达式解析基础。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#155 最小栈 [Medium] Min Stack

模式：栈
思路：主栈每个元素同时存当前最小值，或额外 minStack。
复杂度： $O(1) / O(n)$
易错：重复最小值必须正确保留。
扩展：Max Stack。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#394 字符串解码 [Medium] Decode String

模式：栈
思路：遇 '[' 保存当前字符串与 repeat，遇 ']' 弹出并拼接。
复杂度： $O(\text{输出长度}) / O(n)$
易错：多位数字与嵌套括号。
扩展：表达式/括号递归下降解析。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#739 每日温度 [Medium] Daily Temperatures

模式：单调栈
思路：栈存未找到更高温度的下标并保持单调递减，新温度更大时持续弹栈结算距离。
复杂度： $O(n) / O(n)$
易错：答案是下标差。
扩展：Next Greater Element 系列。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#84 柱状图中最大的矩形 [Hard] Largest Rectangle in Histogram

模式：单调栈

思路：单调递增栈；遇更低高度时弹出柱子，以当前 i 为右边界计算最大矩形。

复杂度： $O(n) / O(n)$

易错：宽度 = right-left-1；加哨兵最稳。

扩展：最大矩形矩阵。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#215 数组中的第 K 个最大元素 [Medium] Kth Largest Element in an Array

模式：堆/选择

思路：维护大小 k 的小顶堆；或 QuickSelect 平均 $O(n)$ 。

复杂度： $O(n \log k) / O(n)$

易错：第 k 大对应小顶堆堆顶。

扩展：Top K、快速选择。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#347 前 K 个高频元素 [Medium] Top K Frequent Elements

模式：堆/桶

思路：先计频，再用大小 k 的小顶堆或按频次桶排序。

复杂度： $O(n \log k) / O(n)$

易错：Go heap 比较函数方向易写反。

扩展：高频词、流式 Top K。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#295 数据流的中位数 [Hard] Find Median from Data Stream

模式：堆

思路：大顶堆存较小一半，小顶堆存较大一半，保持数量差 ≤ 1 且 $\maxLow \leq \minHigh$ 。

复杂度：add $O(\log n)$, find $O(1)$

易错：Go 大顶堆可存负数简化。

扩展：滑动窗口中位数需延迟删除。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#121 买卖股票的最佳时机 [Easy] Best Time to Buy and Sell Stock

模式：贪心
思路：遍历维护历史最低价 minPrice，并更新 price-minPrice 最大值。
复杂度： $O(n) / O(1)$
易错：必须先更新利润还是 min 都可，但要理解。
扩展：股票 II/含冷冻期/手续费 -> DP。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#55 跳跃游戏 [Medium] Jump Game

模式：贪心
思路：维护当前能到达的最远位置 farthest；若 $i > \text{farthest}$ 则失败。
复杂度： $O(n) / O(1)$
易错：关注覆盖范围而非具体路径。
扩展：Jump Game II。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#45 跳跃游戏 II [Medium] Jump Game II

模式：贪心
思路：把一次跳跃看作一层区间；扫描当前层，维护下一层最远端，到 currentEnd 时步数+1。
复杂度： $O(n) / O(1)$
易错：最后一个位置无需再跳。
扩展：BFS 分层的贪心等价。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#763 划分字母区间 [Medium] Partition Labels

模式：贪心
思路：先记录每字符最后位置，扫描时扩展当前段 end； $i = \text{end}$ 时切段。
复杂度： $O(n) / O(\text{字符集})$
易错：段终点由区间内所有字符最后位置共同决定。
扩展：区间合并思想。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#70 爬楼梯 [Easy] Climbing Stairs

模式：DP
思路：dp[i]=dp[i-1]+dp[i-2]，可滚动变量优化空间。
复杂度：O(n) / O(1)
易错：n=1/2 初始化。
扩展：Fibonacci、矩阵快速幂。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#118 杨辉三角 [Easy] Pascal's Triangle

模式：DP
思路：每行两端为 1，中间等于上一行相邻两数之和。
复杂度：O(n^2) / O(n^2)
易错：下标边界。
扩展：只求第 k 行可 O(k) 空间。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#198 打家劫舍 [Medium] House Robber

模式：DP
思路：dp[i]=max(dp[i-1], dp[i-2]+nums[i])。
复杂度：O(n) / O(1)
易错：状态语义要明确“到 i 为止”。
扩展：环形、树形打家劫舍。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#279 完全平方数 [Medium] Perfect Squares

模式：DP
思路：完全背包：dp[i]=min(dp[i-j*j]+1)。
复杂度：O(n sqrt n) / O(n)
易错：初始化 inf，dp[0]=0。
扩展：BFS 最短路、数论四平方定理。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#322 零钱兑换 [Medium] Coin Change

模式：DP

思路：完全背包最小值：对 amount 从 coin 到 target 正向更新。

复杂度： $O(\text{amount} * \text{coins}) / O(\text{amount})$

易错：无解返回 -1；循环方向与 0/1 背包不同。

扩展：组合数 Coin Change II。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#139 单词拆分 [Medium] Word Break

模式：DP

思路：dp[i] 表示前 i 字符可拆；枚举 j<i 且 dp[j] && s[j:i] in dict。

复杂度： $O(n^2) / O(n)$

易错：字符串切片成本；可用 Trie 优化。

扩展：输出所有拆分结果需回溯。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#300 最长递增子序列 [Medium] Longest Increasing Subsequence

模式：DP/二分

思路：维护 tails；tails[len-1] 是该长度递增子序列的最小尾值，用 lower_bound 替换。

复杂度： $O(n \log n) / O(n)$

易错：tails 不一定是真实 LIS。

扩展：恢复具体 LIS 需 predecessor。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#152 乘积最大子数组 [Medium] Maximum Product Subarray

模式：DP

思路：同时维护 maxProd/minProd，因为负数会交换最大最小。

复杂度： $O(n) / O(1)$

易错：遇负数时最大最小角色互换。

扩展：最大子数组和的乘法版。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#416 分割等和子集 [Medium] Partition Equal Subset Sum

模式：0/1 背包

思路：总和为奇数直接 false；目标 sum/2，0/1 背包倒序更新可达性。

复杂度： $O(n \cdot \text{sum}) / O(\text{sum})$

易错：倒序！正序会重复使用元素。

扩展：目标和、子集和。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#32 最长有效括号 [Hard] Longest Valid Parentheses

模式：DP/栈

思路：栈存最后一个无法匹配位置；每次匹配成功长度=i-stackTop。

复杂度： $O(n) / O(n)$

易错：初始化栈 [-1] 很关键。

扩展：DP 解法也常考。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#62 不同路径 [Medium] Unique Paths

模式：二维DP

思路： $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ ，可压缩到一维。

复杂度： $O(mn) / O(n)$

易错：首行首列初始化为 1。

扩展：组合数学 $C(m+n-2, m-1)$ 。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#64 最小路径和 [Medium] Minimum Path Sum

模式：二维DP

思路： $dp[i][j] = grid[i][j] + \min(\text{top}, \text{left})$ ，可原地或一维压缩。

复杂度： $O(mn) / O(n)$

易错：边界第一行/列。

扩展：路径重建。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#5 最长回文子串 [Medium] Longest Palindromic Substring

模式：二维DP/中心扩展
思路：中心扩展最简：每个位置尝试奇/偶中心并更新最长。
复杂度： $O(n^2)$ / $O(1)$
易错：偶数中心别漏。
扩展：Manacher $O(n)$ 。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#1143 最长公共子序列 [Medium] Longest Common Subsequence

模式：二维DP
思路：字符相等 $dp[i][j]=dp[i-1][j-1]+1$ ，否则 $\max(\text{top}, \text{left})$ 。
复杂度： $O(mn)$ / $O(mn)$
易错：子序列非子串。
扩展：恢复 LCS、滚动数组。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#72 编辑距离 [Medium] Edit Distance

模式：二维DP
思路： $dp[i][j]$ 表示 word1 前 i 到 word2 前 j 的最少操作；取插入/删除/替换最小。
复杂度： $O(mn)$ / $O(mn)$
易错：初始化 $dp[i][0]=i$, $dp[0][j]=j$ 。
扩展：带权编辑距离、diff。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#136 只出现一次的数字 [Easy] Single Number

模式：技巧/位运算
思路：所有数异或，成对数字抵消。
复杂度： $O(n)$ / $O(1)$
易错：理解 $x \oplus x = 0$, $x \oplus 0 = x$ 。
扩展：只出现一次 II/III。
双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#169 多数元素 [Easy] Majority Element

模式：技巧/投票

思路：Boyer-Moore：相同票数+1，不同-1，归零换候选。

复杂度： $O(n) / O(1)$

易错：题目保证多数存在；否则需二次验证。

扩展：超过 $n/3$ 的元素。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#75 颜色分类 [Medium] Sort Colors

模式：技巧/三指针

思路：Dutch National Flag：0 放左、2 放右、1 留中间。

复杂度： $O(n) / O(1)$

易错：交换到右侧的新值还未检查，因此 i 不前进。

扩展：三路快排分区。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#31 下一个排列 [Medium] Next Permutation

模式：技巧

思路：从右找首个下降位置 i；再找右侧刚好更大的 j 交换；最后反转后缀。

复杂度： $O(n) / O(1)$

易错：找的是最右 i 且 $nums[i] < nums[i+1]$ 。

扩展：排列字典序算法。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

#287 寻找重复数 [Medium] Find the Duplicate Number

模式：技巧/Floyd

思路：把 $nums[i]$ 看成 next 指针，问题变成环入口；用 Floyd 找入口。

复杂度： $O(n) / O(1)$

易错：数组值范围使映射合法，不能修改数组。

扩展：Cycle II 的数组版本；二分计数解法。

双语言提醒：Go 注意 slice/map/指针/heap.Interface；Python 注意可变对象、tuple/hash、heapq/deque 与切片成本。

五、60 天复习路径

原则：第一轮“按模式学”，第二轮“语言互换 + 闭卷”，最后“混合限时”。不要每天只固定一种语言，否则面试时切换成本会很高。

天数	任务	语言策略	完成标准
Day 1	#1 两数之和；#49 字母异位词分组；#128 最长连续序列	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 2	#283 移动零；#11 盛最多水的容器；#15 三数之和	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 3	#42 接雨水；#3 无重复字符的最长子串；#438 找到字符串中所有字母异位词	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 4	#560 和为 K 的子数组；#239 滑动窗口最大值；#76 最小覆盖子串	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 5	#53 最大子数组和；#56 合并区间；#189 轮转数组	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 6	#238 除自身以外数组的乘积；#41 缺失的第一个正数；#73 矩阵置零	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 7	#54 螺旋矩阵；#48 旋转图像；#240 搜索二维矩阵 II	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 8	#160 相交链表；#206 反转链表；#234 回文链表	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 9	#141 环形链表；#142 环形链表 II；#21 合并两个有序链表	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 10	#2 两数相加；#19 删除链表倒数第 N 个结点；#24 两两交换链表中的节点	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 11	#25 K 个一组翻转链表；#138 随机链表的复制；#148 排序链表	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 12	#23 合并 K 个升序链表；#146 LRU 缓存；#94 二叉树的中序遍历	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 13	#104 二叉树的最大深度；#226 翻转二叉树；#101 对称二叉树	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 14	#543 二叉树的直径；#102 二叉树的层序遍历；#108 将有序数组转换为二叉搜索树	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 15	#98 验证二叉搜索树；#230 二叉搜索树中第 K 小的元素；#199 二叉树的右视图	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 16	#114 二叉树展开为链表；#105 从前序与中序遍历序列构造二叉树；#437 路径总和 III	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 17	#236 二叉树的最近公共祖先；#124 二叉树中的最大路径和；#200 岛屿数量	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 18	#994 腐烂的橘子；#207 课程表；#208 实现 Trie	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 19	#46 全排列；#78 子集；#17 电话号码的字母组合	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 20	#39 组合总和；#22 括号生成；#79 单词搜索	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因

天数	任务	语言策略	完成标准
Day 21	#131 分割回文串；#51 N 皇后	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 22	#35 搜索插入位置；#74 搜索二维矩阵	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 23	#34 在排序数组中查找元素的第一个和最后一个位置；#33 搜索旋转排序数组	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 24	#153 寻找旋转排序数组中的最小值；#4 寻找两个正序数组的中位数	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 25	#20 有效的括号；#155 最小栈	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 26	#394 字符串解码；#739 每日温度	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 27	#84 柱状图中最大的矩形；#215 数组中的第 K 个最大元素	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 28	#347 前 K 个高频元素；#295 数据流的中位数	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 29	#121 买卖股票的最佳时机；#55 跳跃游戏	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 30	#45 跳跃游戏 II；#763 划分字母区间	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 31	#70 爬楼梯；#118 杨辉三角	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 32	#198 打家劫舍；#279 完全平方数	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 33	#322 零钱兑换；#139 单词拆分	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 34	#300 最长递增子序列；#152 乘积最大子数组	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 35	#416 分割等和子集；#32 最长有效括号	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 36	#62 不同路径；#64 最小路径和	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 37	#5 最长回文子串；#1143 最长公共子序列	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 38	#72 编辑距离；#136 只出现一次的数字	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 39	#169 多数元素；#75 颜色分类	奇数题 Go、偶数题 Python；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 40	#31 下一个排列；#287 寻找重复数	奇数题 Python、偶数题 Go；当天最难 1 题换语言重写	独立 AC + 口述思路/复杂度/边界；记录 1 个错误原因
Day 41	二刷 哈希+双指针: #1、#49、#128、#283、#11、#15、#42	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 42	二刷 滑动+前缀: #3、#438、#560、#239、#76	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 43	二刷 数组+矩阵: #53、#56、#189、#238、#41、#73、#54、#48、#240	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59

天数	任务	语言策略	完成标准
Day 44	二刷 链表 I: #160、#206、#234、#141、#142、#21、#2	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 45	二刷 链表 II: #19、#24、#25、#138、#148、#23、#146	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 46	二刷 二叉树 I: #94、#104、#226、#101、#543、#102、#108、#98	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 47	二刷 二叉树 II: #230、#199、#114、#105、#437、#236、#124	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 48	二刷 图+回溯: #200、#994、#207、#208、#46、#78、#17、#39、#22、#79、#131、#51	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 49	二刷 二分+栈+堆: #35、#74、#34、#33、#153、#4、#20、#155、#394、#739、#84、#215、#347、#295	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 50	二刷 贪心+DP+技巧: #121、#55、#45、#763、#70、#118、#198、#279、#322、#139、#300、#152、#416、#32、#62、#64、#5、#1143、#72、#136、#169、#75、#31、#287	与第一轮使用相反语言；只看题目不看笔记	每题 15-25 分钟；不会的标红进入 Day 59
Day 51	限时混合 Set A：哈希/双指针/滑窗/数组，共 5 题	Go 3 + Python 2	90 分钟；每题先写思路再编码
Day 52	限时混合 Set B：链表，共 4 题	Python 2 + Go 2	重点：dummy、反转、快慢指针
Day 53	限时混合 Set C：二叉树，共 5 题	Go 3 + Python 2	先判断 DFS/BFS 和返回值语义
Day 54	限时混合 Set D：图/回溯/二分，共 5 题	Python 3 + Go 2	强调 visited、回滚、边界二分
Day 55	限时混合 Set E：栈/堆/贪心/DP，共 5 题	Go 3 + Python 2	必须写复杂度与循环方向
Day 56	Mock Interview #1：2 题，45+45 分钟	第一题 Go，第二题 Python	像面试一样口述、测试样例、复杂度
Day 57	Mock Interview #2：2 题，45+45 分钟	第一题 Python，第二题 Go	重点练边界与 debug 表达
Day 58	Hard Day：从 42/25/124/4/84/295 任选 4 题	2 Go + 2 Python	不追求全 AC，追求拆解和证明
Day 59	弱项日：只做前 58 天标红题	全部使用较弱语言	把错误归类：识别错/边界错/实现错/复杂度错
Day 60	Final Review：15 母题 + 14 Pattern + 语言速查	闭卷口述，任选 3 题现场写	目标：看到题能在 2 分钟内选出模式与模板

六、错题不要只记答案：按错误类型复盘

错误类型	表现	修复动作
Pattern 识别错	把前缀和写成滑窗；把多源最短路写成 DFS	重画输入特征：连续区间？单调性？依赖图？“下一个更大”？
不变量没想清	双指针移动方向靠猜；贪心无法解释	编码前先用一句话写“循环结束时我保证什么”。
边界错误	二分死循环、链表删头、矩阵单行重复	固定使用一种区间定义；链表优先 dummy；手测空/1/2 个元素。
语言实现错	Go map 零值误判；Python list 当队列；path 未复制	维护 Go/Python 语言坑清单，二刷时刻意换语言。
复杂度错	窗口里反复排序；每个节点重新扫描；字符串切片过多	写完后问：每个元素最多被访问/入栈/出栈几次？

最终自测 Checklist

- ☐ 能否在 2 分钟内把 100 题归入 14 个模式之一？
- ☐ 能否闭卷写出 Go/Python 的哈希、队列、堆、DFS/BFS、二分、回溯、DP 基础模板？
- ☐ 是否能解释 3Sum 去重、接雨水双指针、旋转数组二分、最大路径和返回值的正确性？
- ☐ 是否能在 Go 中熟练处理 slice、map、pointer、container/heap？
- ☐ 是否能在 Python 中熟练使用 dict/set/deque/heapq，并避免可变对象共享与 O(n) 队列操作？
- ☐ 每个 Hard 题是否至少知道一个可接受解法与一个常见替代解法？
- ☐ 是否积累了自己的“边界测试 5 件套”：空、1 个、2 个、全相同、极端值？

最后建议：不要追求“100 题代码全部背下来”。真正的目标是把它们压缩成十几个 Pattern，并且能在 Go 与 Python 之间无痛映射。